

Инструменты для профилирования проблемных сценариев на Android

Ускорение сценариев - это часто нетривиальная задача и помочь найти место для ускорения могут различные инструменты.

Ликбез про анализ производительности

Анализ производительности приложения — это процесс оценки того, насколько эффективно приложение использует системные ресурсы, используя для этого различные метрики и инструменты. Цель анализа — выявить и устранить проблемы и узкие места, гарантируя, что приложение работает быстро, стабильно и обеспечивает положительный опыт для пользователя.

Ключевые метрики

- **Время отклика:** как быстро приложение реагирует на действия пользователя;
- **Время загрузки:** как быстро приложение запускается и загружает все необходимые данные;
- **Частота ошибок:** как часто приложение сталкивается с ошибками, которые могут привести к сбоям;
- **Использование ресурсов:**
 - Использование CPU/GPU
 - Использование памяти
 - Использование сети
 - Использование батареи

Инструменты и методы анализа

- **APM-системы (Application Performance Monitoring):**

Используют телеметрические данные для мониторинга производительности в режиме реального времени и анализа проблем.
- **Бенчмарки:**

Программы, которые выполняют тесты и выдают оценку производительности, особенно для мобильных устройств.
- **Инструменты нагрузочного тестирования:**

Используются для проверки, как приложение ведет себя под высокой нагрузкой.
- **Профилировщики кода:**

Позволяют детально анализировать, как код приложения расходует ресурсы.

Этапы анализа производительности

1. **Сбор данных:** с помощью APM-систем, профилировщиков или других инструментов собираются данные о времени отклика, использовании ресурсов и ошибках.
2. **Анализ метрик:** собранные данные анализируются, чтобы выявить "узкие места" и области, где производительность ниже ожидаемой.
3. **Оптимизация:** на основе анализа вносятся изменения в код или инфраструктуру для повышения производительности.
4. **Тестирование:** проводится повторное тестирование, чтобы убедиться в эффективности внесенных изменений.

Анализ производительности в плоскости Android

За последние годы Google подготовила хороший [раздел](#) по производительности приложений в официальной документации, а также предоставляет своеобразный [RoadMap](#) для заинтересованных разработчиков.

Базовые метрики с точки зрения Google

- **App startup**
- **Slow rendering**

- Screen transitions and navigation events

Инструменты и методы анализа

- АРМ-системы: [Android vitals](#), [Firebase performance](#)
- Бенчмарки: [Macrobenchmark](#), [Microbenchmark](#)
- Профилировщики: [Android Studio Profiles](#), [SimplePerf](#), [Perfetto](#).

Какой метод профилирования использовать, очень хорошо разобрано в [документации](#).

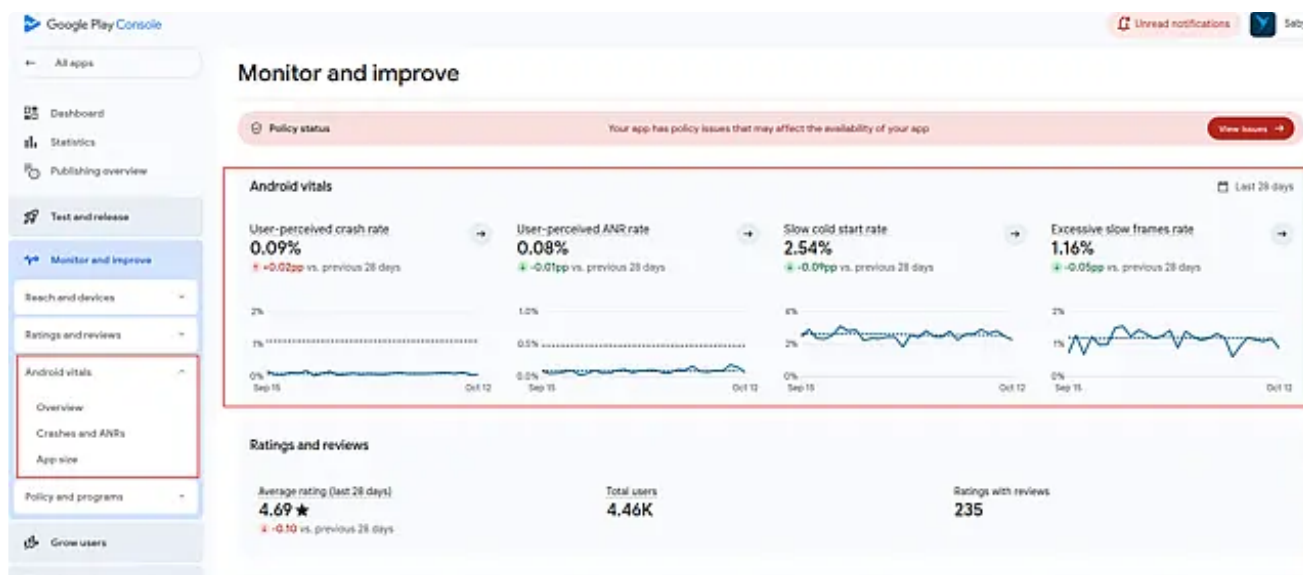
Использование инструментов на практике

В качестве подопытного выбрано приложение **Saby**

АРМ

Android vitals

В *Google Play Console* можно найти отыскать специальную секцию с метриками, которые *Google* собирает автоматически без нашей помощи.



Что интересного можно отсюда подчерпнуть?

Анализ старта приложения

- Сводная информация по разным типам старта приложения.

Подробнее про варианты старта приложения [тут](#).

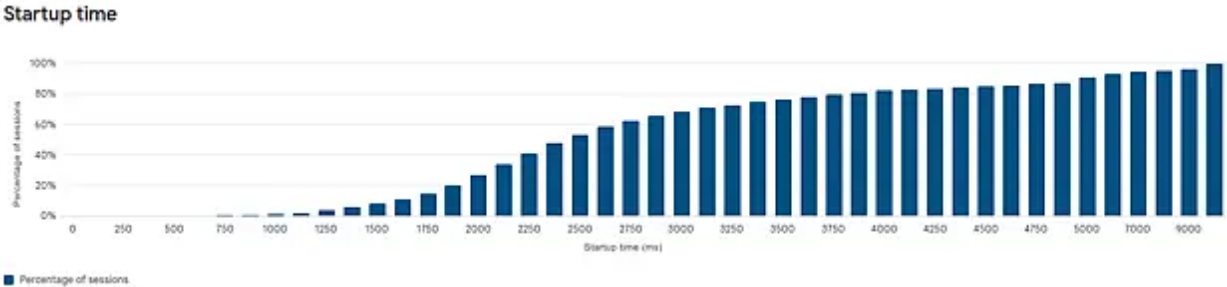
Start-up and loading times

Start-up time (time to initial display)

The time it takes from when a user launches your app, to when the first frames appear on the screen. This is also known as 'time to initial display'. Your app may not be ready for the user to start interacting with it after this time, for example, if your app has additional loading screens. [Learn more](#)

Metric	Last 28 days	vs. previous 28 days	vs. peers' median
Slow cold start ⓘ	2.54%	-0.10% +	+2.23% + →
Slow warm start ⓘ	8.10%	-0.64% -	+5.96% + →
Slow hot start ⓘ	2.18%	-0.08% -	+1.67% + →

- Кластеризованное время старта



Анализ отрисовки экранов

- Сводная информация

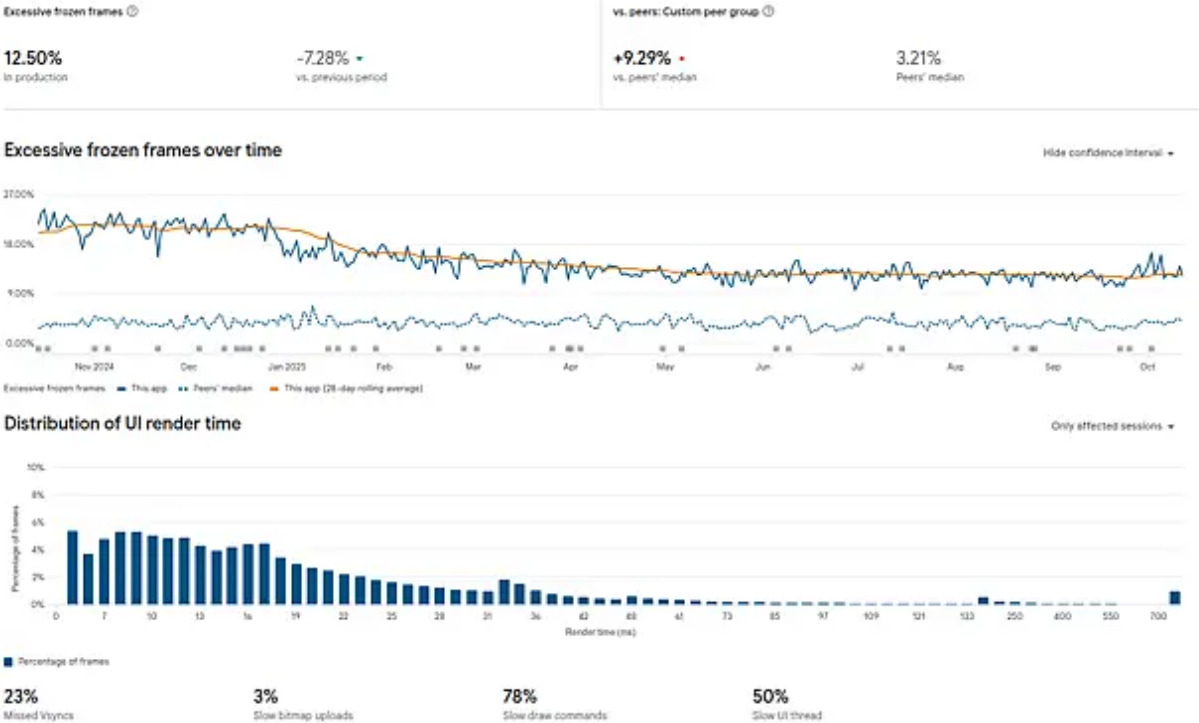
Rendering

Android UI toolkit rendering

Frames rendered that use built-in Android UI components. For example menus, or embedded ads. [Learn more](#)

Metric	Last 28 days	vs. previous 28 days	vs. peers' median
Excessive slow frames ⓘ	1.16%	-0.04% +	+0.79% + →
Excessive frozen frames ⓘ	12.50%	+0.27% -	+9.29% + →

- Кластеризованная информация



Анализ размера приложения

- Размер приложения - удобно это получать от Google, учитывая что им мы поставляем [app bundle](#).

App download size ⓘ

Size ⓘ

141 MB

For reference device

130 MB – 143 MB

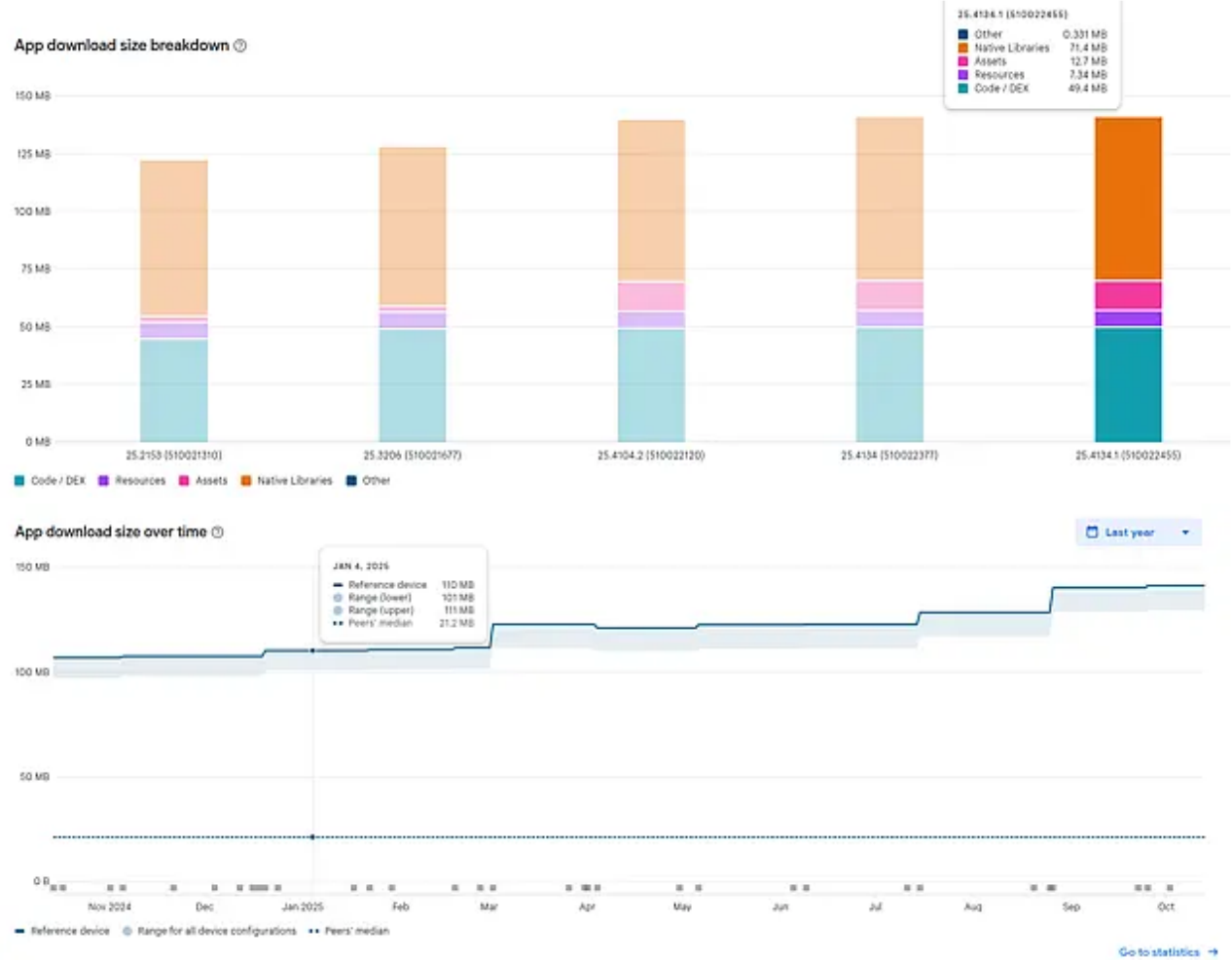
Across all device configurations

- Чек-лист по возможным оптимизациям

App size optimizations

Recommendation	Potential saving	
Extract large files	–	
Enable screen density configuration APKs	–	Implemented
Enable ABI configuration APKs	–	Implemented
Enable code shrinking and obfuscation	–	Implemented

- Изменение веса приложения по версиям



Аварийные падения приложения (краши)

Crashes and ANRs

Prioritize and debug specific stability issues in your app or game. [Show more](#)

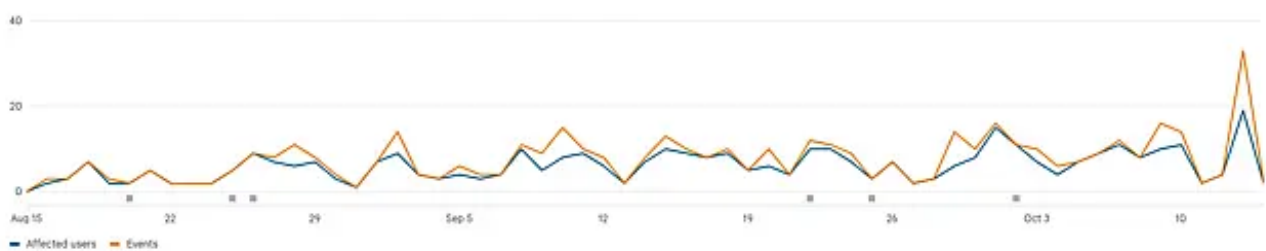
Issues

Issue category: All Type: User-perceived crashes X Add filter X

Last 60 days

Affected users

Aug 15 – Oct 14, 2025



Общая картина


```

(std::__ndk1::condition_variable::wait(std::__ndk1::unique_lock<std::__ndk1::
mutex>&)+24) (BuildId: b04675a35ad96f8a9dcaa073e3bd31d4536f00ad)
#04 pc 0x00000000003dd0438
/data/app/~~HhLElq0J7vdmhfDMok4rTQ==/ru.tensor.sbis.droid.saby-
vtluD3kAHZj3IRkKItn5Fw==/lib/arm64/libsbis-app-controller.so
(sbis::Condition::Wait()) (BuildId: d55e4ca572e57cef)
#05 pc 0x00000000005101080
/data/app/~~HhLElq0J7vdmhfDMok4rTQ==/ru.tensor.sbis.droid.saby-
vtluD3kAHZj3IRkKItn5Fw==/lib/arm64/libsbis-app-controller.so
(sbis::sync::SyncManagerImpl::WaitForCompletionOf(std::__ndk1::unordered_set<
std::__ndk1::basic_string<wchar_t, std::__ndk1::char_traits<wchar_t>,
std::__ndk1::allocator<wchar_t>>,
std::__ndk1::hash<std::__ndk1::basic_string<wchar_t,
std::__ndk1::char_traits<wchar_t>, std::__ndk1::allocator<wchar_t>>>,
std::__ndk1::equal_to<std::__ndk1::basic_string<wchar_t,
std::__ndk1::char_traits<wchar_t>, std::__ndk1::allocator<wchar_t>>>,
std::__ndk1::allocator<std::__ndk1::basic_string<wchar_t,
std::__ndk1::char_traits<wchar_t>, std::__ndk1::allocator<wchar_t>>>>
const&)) (BuildId: d55e4ca572e57cef)
#06 pc 0x00000000005100fcc
/data/app/~~HhLElq0J7vdmhfDMok4rTQ==/ru.tensor.sbis.droid.saby-
vtluD3kAHZj3IRkKItn5Fw==/lib/arm64/libsbis-app-controller.so
(sbis::sync::SyncManager::WaitForCompletionOf(std::__ndk1::basic_string<wchar
_t, std::__ndk1::char_traits<wchar_t>, std::__ndk1::allocator<wchar_t>>
const&)) (BuildId: d55e4ca572e57cef)
#07 pc 0x000000000050ff378
/data/app/~~HhLElq0J7vdmhfDMok4rTQ==/ru.tensor.sbis.droid.saby-
vtluD3kAHZj3IRkKItn5Fw==/lib/arm64/libsbis-app-controller.so
(sbis::sync::SyncManagerImpl::RemoveSync(std::__ndk1::basic_string<wchar_t,
std::__ndk1::char_traits<wchar_t>, std::__ndk1::allocator<wchar_t>> const&))
(BuildId: d55e4ca572e57cef)
#08 pc 0x00000000007ea10a0
/data/app/~~HhLElq0J7vdmhfDMok4rTQ==/ru.tensor.sbis.droid.saby-
vtluD3kAHZj3IRkKItn5Fw==/lib/arm64/libsbis-app-controller.so (non-virtual
thunk to
sbis::search::HierarchySearchCollection<sbis::service::detail::HierarchyColle
ction<sbis::mobile::mydoc::MyDocsViewModel, false,
sbis::mobile::mydoc::djinni_traits::HierarchyCollectionOfMyDocsViewModelTrait
s>, sbis::service::HierarchyDataSource<sbis::mobile::mydoc::MyDocsRawModel,
sbis::mobile::mydoc::MyDocsFilter,
sbis::service::HierarchyPagination<sbis::IdentifierType<boost::uuids::uuid>,
sbis::mobile::mydoc::MyDocsAnchor, false>,
sbis::service::PathModel<sbis::IdentifierType<boost::uuids::uuid>,
std::__ndk1::unordered_map<std::__ndk1::basic_string<wchar_t,
std::__ndk1::char_traits<wchar_t>, std::__ndk1::allocator<wchar_t>>,
std::__ndk1::basic_string<wchar_t, std::__ndk1::char_traits<wchar_t>,
std::__ndk1::allocator<wchar_t>>,
std::__ndk1::hash<std::__ndk1::basic_string<wchar_t,
std::__ndk1::char_traits<wchar_t>, std::__ndk1::allocator<wchar_t>>>,
std::__ndk1::equal_to<std::__ndk1::basic_string<wchar_t,
std::__ndk1::char_traits<wchar_t>, std::__ndk1::allocator<wchar_t>>>,
std::__ndk1::allocator<std::__ndk1::pair<std::__ndk1::basic_string<wchar_t,
std::__ndk1::char_traits<wchar_t>, std::__ndk1::allocator<wchar_t>> const,
std::__ndk1::basic_string<wchar_t, std::__ndk1::char_traits<wchar_t>,
std::__ndk1::allocator<wchar_t>>>>>, false>, void,

```

```

std::__ndk1::vector<boost::uuids::uuid,
std::__ndk1::allocator<boost::uuids::uuid>> const&>,
sbis::mobile::mydoc::MyDocsSearcherImpl, 16>::Dispose()) (BuildId:
d55e4ca572e57cef)
    #09 pc 0x00000000007e9eb98
/data/app/~~HhLElq0J7vdmhfDMok4rTQ==/ru.tensor.sbis.droid.saby-
vtluD3kAHZj3IRkKItn5Fw==/lib/arm64/libsbis-app-controller.so
(sbis::service::CollectionStorage<sbis::service::detail::CollectionStorage<sb
is::mobile::mydoc::MyDocsViewModel, sbis::mobile::mydoc::MyDocsFilter, false,
sbis::mobile::mydoc::djinni_traits::CollectionStorageOfMyDocsViewModelMyDocsF
ilterTraits>,
sbis::service::HierarchyCollection<sbis::service::detail::HierarchyCollection
<sbis::mobile::mydoc::MyDocsViewModel, false,
sbis::mobile::mydoc::djinni_traits::HierarchyCollectionOfMyDocsViewModelTrait
s>, sbis::mobile::mydoc::MyDocsControllerImpl, 0>>::Trim()) (BuildId:
d55e4ca572e57cef)
    #10 pc 0x00000000007e9c760
/data/app/~~HhLElq0J7vdmhfDMok4rTQ==/ru.tensor.sbis.droid.saby-
vtluD3kAHZj3IRkKItn5Fw==/lib/arm64/libsbis-app-controller.so
(sbis::service::CollectionStorage<sbis::service::detail::CollectionStorage<sb
is::mobile::mydoc::MyDocsViewModel, sbis::mobile::mydoc::MyDocsFilter, false,
sbis::mobile::mydoc::djinni_traits::CollectionStorageOfMyDocsViewModelMyDocsF
ilterTraits>,
sbis::service::HierarchyCollection<sbis::service::detail::HierarchyCollection
<sbis::mobile::mydoc::MyDocsViewModel, false,
sbis::mobile::mydoc::djinni_traits::HierarchyCollectionOfMyDocsViewModelTrait
s>, sbis::mobile::mydoc::MyDocsControllerImpl,
0>>::ChangeFilter(sbis::mobile::mydoc::MyDocsFilter const&)) (BuildId:
d55e4ca572e57cef)
    #11 pc 0x00000000007efc168
/data/app/~~HhLElq0J7vdmhfDMok4rTQ==/ru.tensor.sbis.droid.saby-
vtluD3kAHZj3IRkKItn5Fw==/lib/arm64/libsbis-app-controller.so
(Java_ru_tensor_sbis_mydoc_generated_CollectionStorageOfMyDocsViewModelMyDocs
Filter_00024CppProxy_native_1changeFilter+300) (BuildId: d55e4ca572e57cef)
    at
ru.tensor.sbis.mydoc.generated.CollectionStorageOfMyDocsViewModelMyDocsFilter
$CppProxy.native_changeFilter
(CollectionStorageOfMyDocsViewModelMyDocsFilter.kt)
    at
ru.tensor.sbis.mydoc.generated.CollectionStorageOfMyDocsViewModelMyDocsFilter
$CppProxy.changeFilter
(CollectionStorageOfMyDocsViewModelMyDocsFilter.kt:114)
    at
ru.tensor.sbis.mydoc.generated.CollectionStorageOfMyDocsViewModelMyDocsFilter
$CppProxy.changeFilter (CollectionStorageOfMyDocsViewModelMyDocsFilter.kt:27)
    at
ru.tensor.sbis.my_documents.collection.MyDocumentsCollectionFactory.createCol
lection (MyDocumentsCollectionFactory.kt:25)
    at
ru.tensor.sbis.my_documents.collection.MyDocumentsCollectionFactory.createCol
lection (MyDocumentsCollectionFactory.kt:14)
    at ru.tensor.sbis.edo.doc.registry.impl.DocRegistryComponentImpl.reset
(DocRegistryComponentImpl.kt:167)
    at
ru.tensor.sbis.edo.doc.registry.impl.DocRegistryComponentImpl.onSearchQueryCh

```



```

anged (DocRegistryComponentImpl.kt:141)
    at
ru.tensor.sbis.edo.doc.registry.shared.DocRegistryComponentView$initTopNavView$1$1$2$1.invoke$lambda$0 (DocRegistryComponentView.kt:209)
    at
ru.tensor.sbis.design.view.input.searchinput.SearchInputController.runTextChangedCallbacksExecute$lambda$2 (SearchInputController.kt:174)
    at android.os.Handler.handleCallback (Handler.java:942)
    at android.os.Handler.dispatchMessage (Handler.java:99)
    at android.os.Looper.loopOnce (Looper.java:201)
    at android.os.Looper.loop (Looper.java:288)
    at android.app.ActivityThread.main (ActivityThread.java:8121)
    at java.lang.reflect.Method.invoke (Native method)
    at com.android.internal.os.RuntimeInit$MethodAndArgsCaller.run (RuntimeInit.java:588)
    at com.android.internal.os.ZygoteInit.main (ZygoteInit.java:1019)

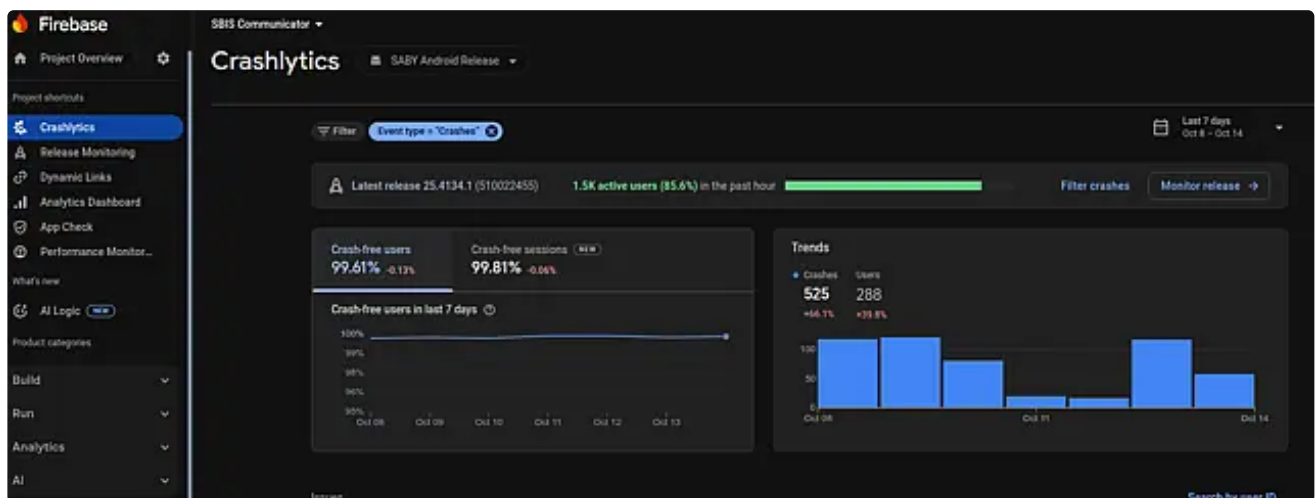
```

Это стек вызовов главного потока. Четко видно, что обращение к CRUD-коллекции на MainThread, с последующим вызовом микросервиса контроллера, привело к зависанию интерфейса пользователя.

🔔 Никогда не обращайтесь к микросервисам на главном потоке!

Firebase

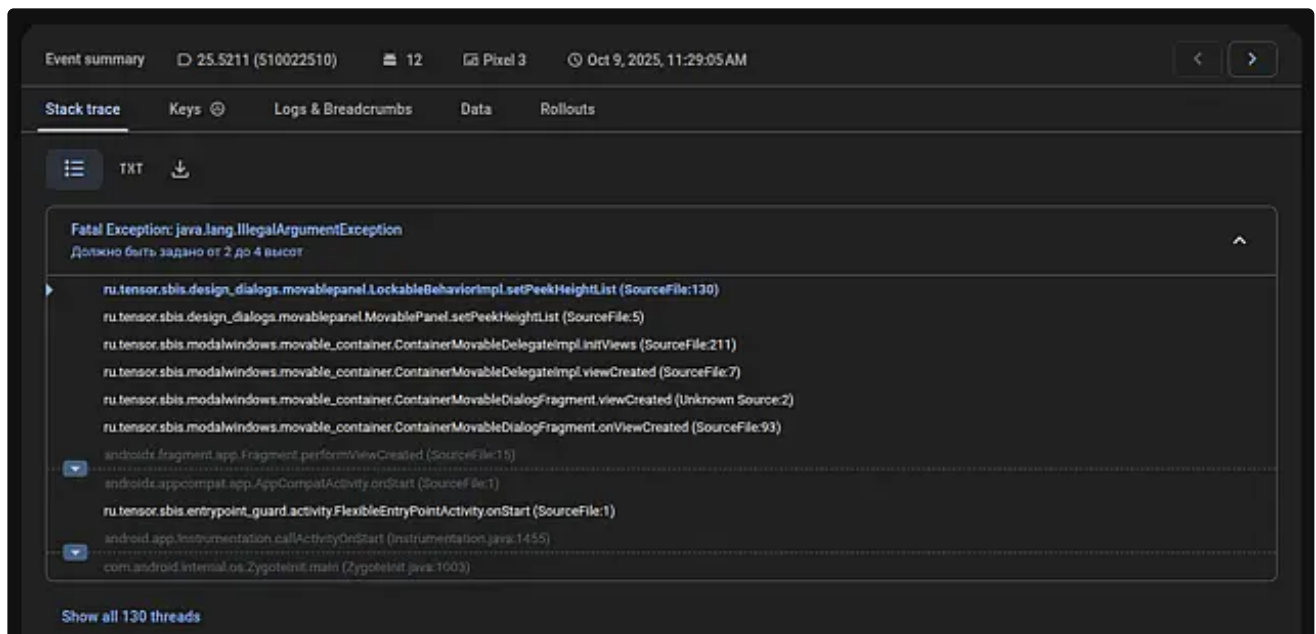
Firebase crashlytics



Это **основной** наш инструмент для разбора **крашей**. А вот ANR у него детектятся **хуже**, чем в Android Vitals и стеки по ним менее подробные. Если в случае Android Vitals система сама собирает краши и ANR без нашего участия, то в случае с Firebase Crashlytics за данную работу отвечает специально подключенная SDK и у нас появляется возможность обогащать отчеты собственными логами и ключами.

Разбор одного из отчетов

Для примера разберем [данный](#) сбой.



Хоть у нас есть место возникновения ошибки, в большинстве случаев важно понимать сценарий. Дополнительную информацию можно найти в ключиках:

- имя процесса приложения (в наших приложениях есть компоненты, которые запускаются в отдельных процессах);
- сервер, с которым взаимодействовало приложения (*prod*, *fix*, *test*);
- время старта приложения и время когда произошел краш;
- замеры времени инициализации контроллера и плагинной системы (важны при анализе ANR и комплексных сбоев);
- время установки приложения и время обновления (помогают распознавать проблемные сценарии с обратной совместимостью при обновлении);
- был ли восстановлен процесс приложения или "холодный" старт;
- было ли приложение в бэграунде;
- включен ли режим оптимизации заряда батареи;
- ключ "*session*" для поиска логов в *sbis.cloud*.

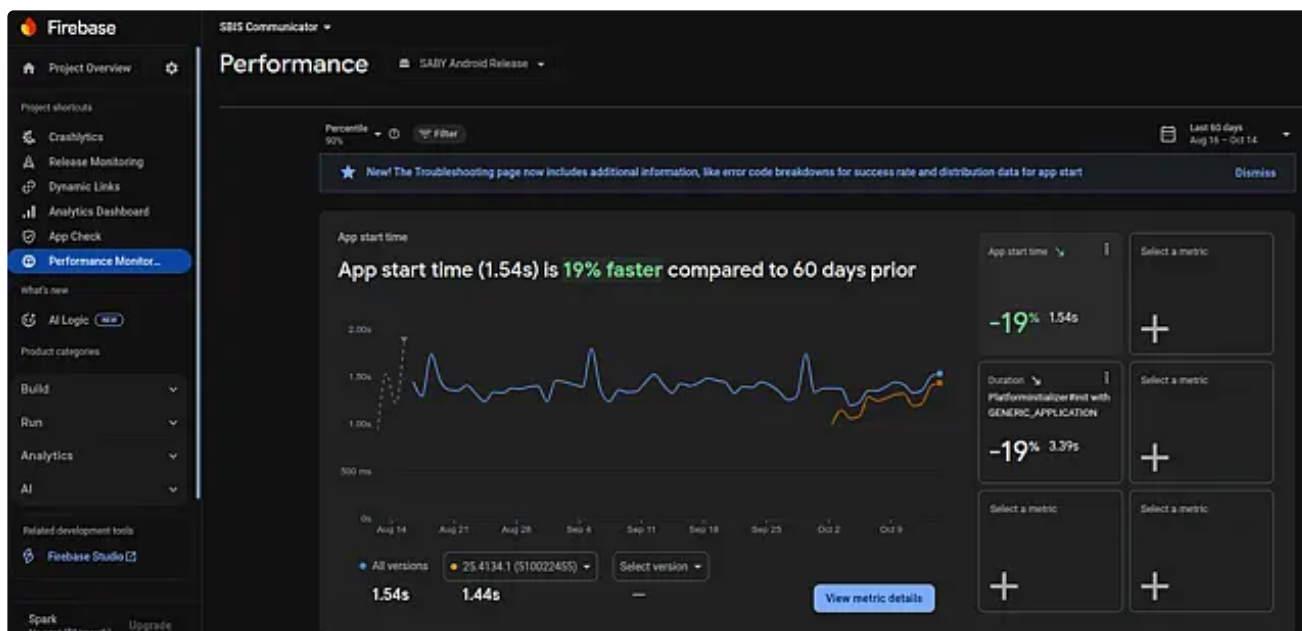
Stack trace			Keys	Logs & Breadcrumbs	Data	Rollouts
Filter keys						
Key	Value for this event	Values across events				
battery_saving_mode_before_initialization	false		Common			
ciBuild	true		Common			
extract_native_libs	true		Common			
in_background	true		Common			
InstallerSource	com.google.android.packageinstaller		Common			
lastDomain	https://fix-online.sbis.ru		Common			
process restored	false		Common			
process started with	ru.tensor.sbis.droid.saby.screen.launch.LaunchActivity		Common			
process_name	ru.tensor.sbis.droid.saby		Common			
app started at	чт окт. 09 11:27:27 GMT+03:00 2025		Common			
controller's init time	1182 ms		Common			
crash time	чт окт. 09 11:29:05 GMT+03:00 2025		Common			
first_install_time	вт окт. 07 15:05:51 GMT+03:00 2025		Common			
last exit info	DateTime: 09.10.25 11:27:10.179; Description: empty #17; PSS: 0; RSS: 0; ProcessName: ru.tensor.sbis.droid.saby		Common			
last_update_time	вт окт. 07 15:05:51 GMT+03:00 2025		Common			
native libs	/data/app/~~IdeahZlEx2bYtF2m0AqZDw==/ru.tensor.sbis.droid.saby-ZfSk7XTsapCEJ15djk_Q==/lib/arm64		Common			
plugin system's init time	109 ms		Common			
session	86536e25-097b-4f12-a43d-e1544e9b54f1		Common			

А также во вкладке логи можно доуточнить сценарий:

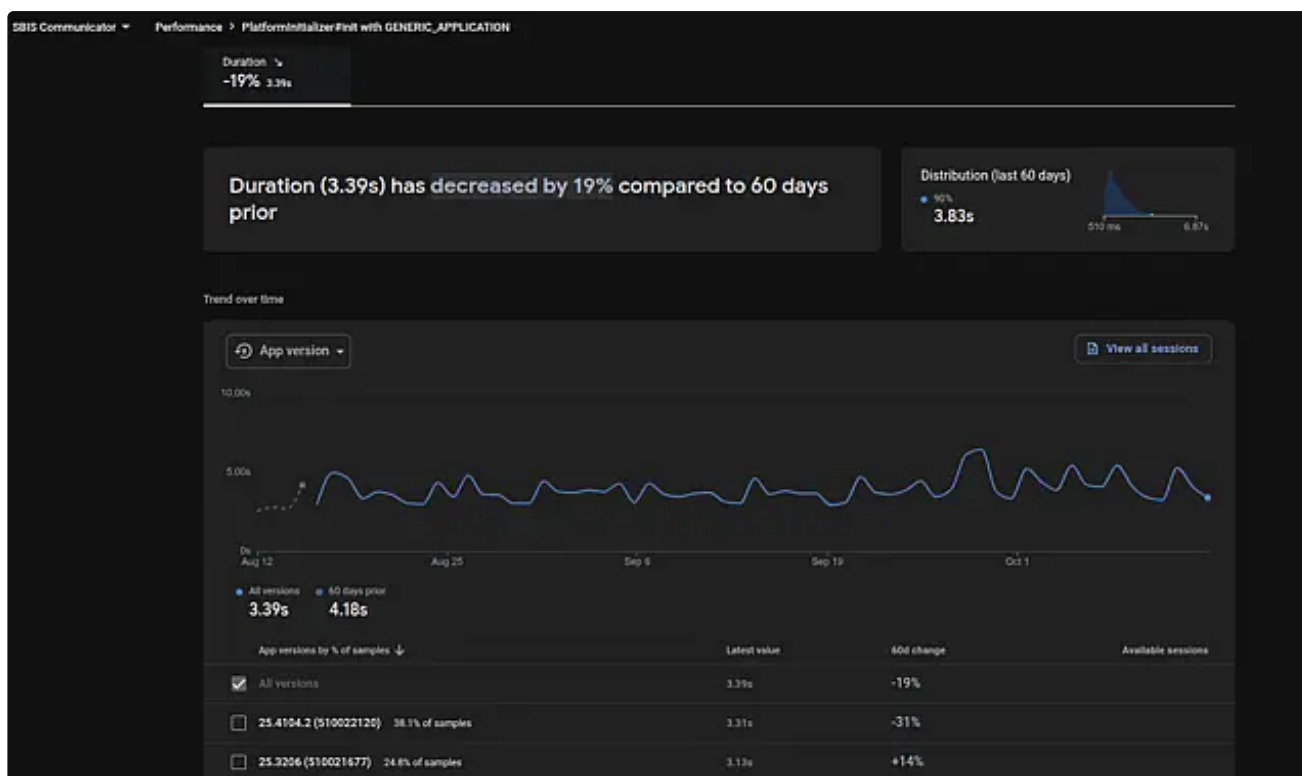
Stack trace			Keys	Logs & Breadcrumbs	Data	Rollouts
Filter logs						
#	Time (newest first)	Source	Log			
1	11:29:04.699 AM	Screen_Name	TaskListFragment(ru.tensor.sbis.droid.saby.screen.main.MainActivity)			
2	11:29:04.698 AM	Screen_Name	FiltersFragment(ru.tensor.sbis.droid.saby.screen.main.MainActivity)			
3	11:29:04.697 AM	Screen_Name	MonthModePeriodPickerFragment(ru.tensor.sbis.droid.saby.screen.main.MainActivity)			
4	11:29:04.689 AM	[EP0]	ru.tensor.sbis.droid.saby.screen.main.MainActivity restored [isRoot=true]			
5	11:29:04.596 AM	[EP0]	ru.tensor.sbis.droid.saby.screen.main.MainActivity waitTerminalOrLockingResult in progress [initState=InitCompleted,progress=ru.tensor.sbis.base_app.initializers.initializers.RequiredUpdateInitializerDecorator\$ProgressStatus\$0,originThread=Thread[DefaultDispatcher-worker-2,5,main]]			
6	11:29:04.482 AM	[EP0]	ru.tensor.sbis.droid.saby.screen.main.MainActivity destroyed [isFinishing=false]			
7	11:29:04.467 AM	[EP0]	ru.tensor.sbis.droid.saby.screen.main.MainActivity save state			
8	11:29:04.444 AM	[EP0]	ru.tensor.sbis.droid.saby.screen.main.MainActivity paused [isFinishing=false]			
9	11:29:02.834 AM	Screen_Name	MonthModePeriodPickerFragment(ru.tensor.sbis.droid.saby.screen.main.MainActivity)			
10	11:29:01.593 AM	Screen_Name	FiltersFragment(ru.tensor.sbis.droid.saby.screen.main.MainActivity)			
11	11:28:59.708 AM	Screen_Name	TaskListFragment(ru.tensor.sbis.droid.saby.screen.main.MainActivity)			

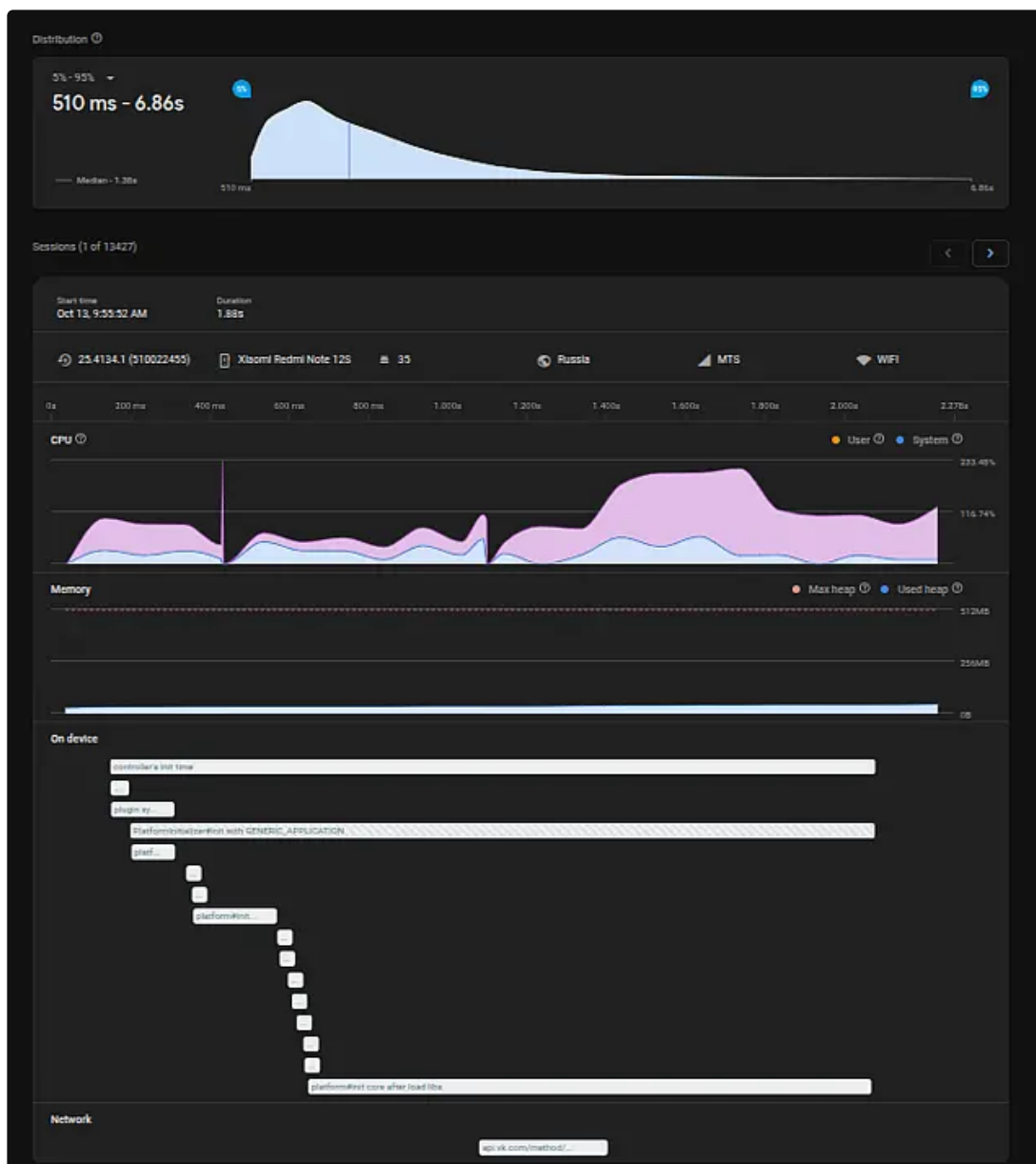
Не всегда удастся досконально прояснить картину, но для решения большинства проблем этой информации хватает.

Firestore performance



Этот инструмент интересен тем, что позволяет выделять собственные метрики и публиковать по ним информацию. Подробнее про это в [официальной](#) документации. В наших приложениях, например, есть метрика на инициализацию контроллера.





Очень показателен именно таймлайн в этом случае, соотнесенные с загрузкой процессора и памяти.

Профилирование

Perfetto

Удобно использовать [Perfetto Tracing](#) для понимания картины в целом. Согласно [документации](#), этот метод профилирования обладает минимальными накладными расходами и позволяет соотносить информацию по загрузке процессора, памяти, потоков приложений (не только инспектируемого, но и соседних) во времени.

На данном шаге легче выявлять узкие места и при необходимости либо размечать код, используя библиотеку [androidx.tracing](#), либо использовать [альтернативные](#) профилировщики.

SimplePerf

[Статья](#) с разбором *SimplePerf* от коллег из платформы. Из неудобств данного способа - информация по потокам никак не выровнена друг от друга (кто и когда стартанул). В некоторых ситуациях это важно. А учитывая, что работа контроллера не видна в *Perfetto* (требуется ручная [инструментация](#)), на текущий момент это единственный способ профилирования контроллера, к сожалению. Пример такого отчета можно посмотреть [тут](#).

Бенчмарк

Google проделала отличную работу и предоставили нам фреймворки для бенчмакинга: микро и макро. По настройке каждого типа все подробно расписано в [документации](#). В качестве пробы пера был создан ряд [модулей](#) на уровне приложения *Saby* - все описанные ниже результаты каждый может воспроизвести локально.



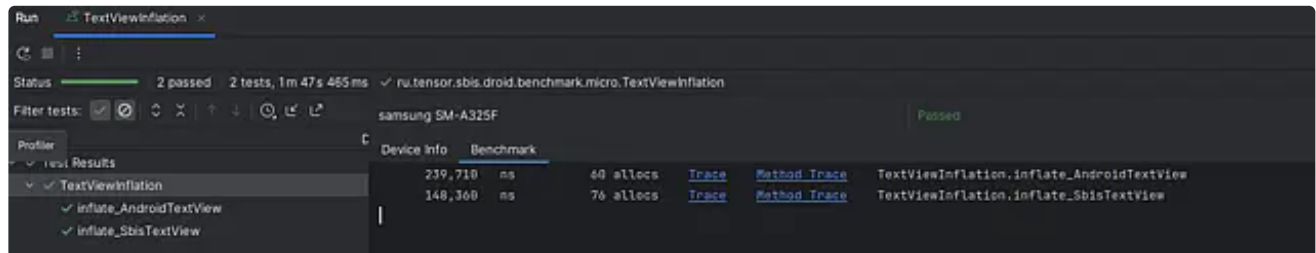
Общая рекомендация: проводить профилирование только на **реализных** сборках и на **реальных** устройствах.
В качестве подопытного устройства взят *Samsung A32* с 3.7Gb RAM.

Давайте посмотрим, как можно применить это в наших реалиях.

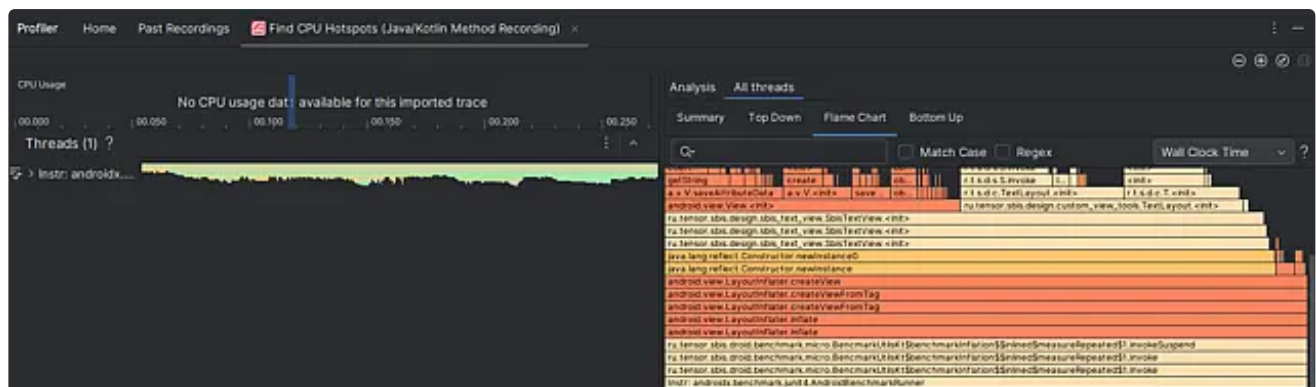
Microbenchmark

Данный тип бенчмаркинга запускает множество замеров в рамках **одного JVM**-процесса. Главная особенность - тесты запускаются в изоляции: **не учитывается** влияние GC и других процессов в системе. Отсюда вытекает основное ограничение этого способа: бесполезно снимать на классах/методах, защищенных от множественного вызова (синглтоны и подобная статика).

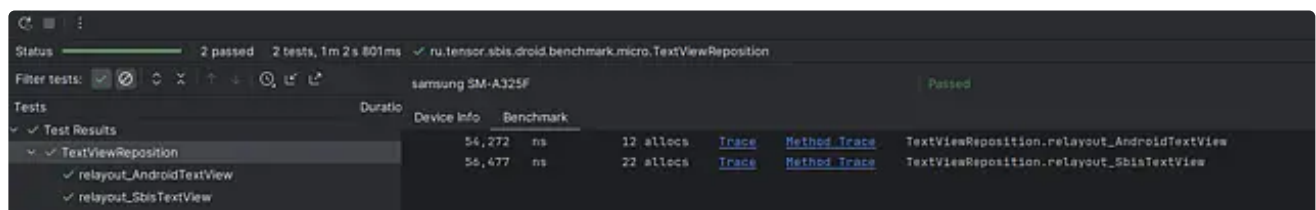
Очень хотелось сделать не просто искусственный тест, а максимально полезный. И ничего лучше в голову не пришло, чем создать тесты для атомарных компонентов нашей дизайн системы. Так появился [сравнительный замер](#) инфлейта *TextView* и *SbisTextView* из xml.



Видно, что коллеги из команды компонентов поработали на славу со *SbisTextView* - ускорили в 1.6 раз. В ходе запуска бенчмарка был сформирован отчет и трейсы для детального анализа.

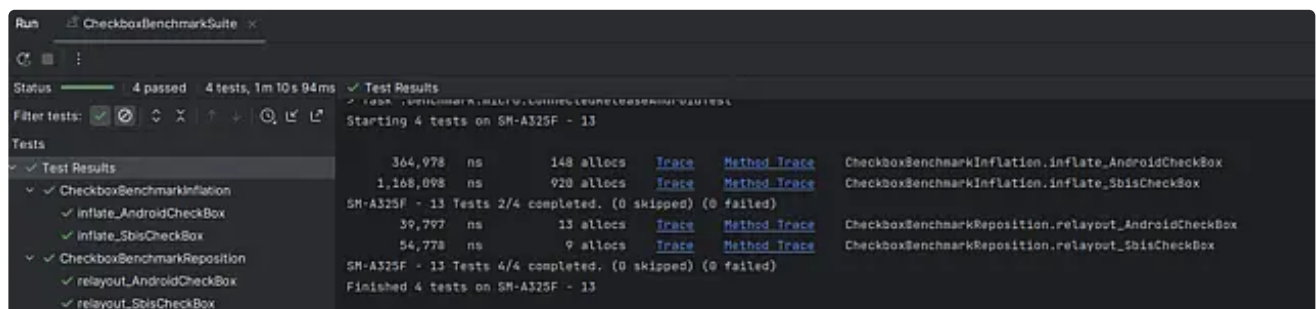


А что если сравнить, как те же компоненты реагируют на смену размеров (onMeasure + onLayout)? Пишем тест, запускаем и получаем отчет:



На уровне! Хотя наш компонент по количеству созданных объектов превысил в 2 раза.

А что с другими базовыми компонентами? Быстро накидываем тесты и смотрим.



Сравнение *SbisCheckBox* и системного *CheckBox*

Run ButtonBenchmarkSuite

Status 4 passed 4 tests, 1m 6s 855ms ✓ Test Results

Filter tests: ✓ ✗ ↕ ✕ ↑ ↓ 🔍 >

Tests

✓ Test Results

- ✓ ButtonBenchmarkInflation
 - ✓ inflate_AndroidButton 356,338 ns 303 allocs [Trace](#) [Method Trace](#) ButtonBenchmarkInflation.inflate_AndroidButton
 - ✓ inflate_SbsButton 580,825 ns 329 allocs [Trace](#) [Method Trace](#) ButtonBenchmarkInflation.inflate_SbsButton
- ✓ ButtonBenchmarkReposition
 - ✓ layout_AndroidButton 61,828 ns 12 allocs [Trace](#) [Method Trace](#) ButtonBenchmarkReposition.layout_AndroidButton
 - ✓ layout_SbsButton 154,958 ns 38 allocs [Trace](#) [Method Trace](#) ButtonBenchmarkReposition.layout_SbsButton

Starting 4 tests on SM-A325F - 13

Finished 4 tests on SM-A325F - 13

> Task :benchmark:micro:createReleaseAndroidTestApkListingFileRedirect UP-TO-DATE

> Task :benchmark:micro:connectedReleaseAndroidTest

Сравнение SbsButton и системного Button

Цифры говорят сами за себя. На основе таких отчетов можно формировать baseline и пробовать оптимизировать с последующим повторным запуском бенчмарков. Кажется, что очень удобный инструмент.

⚠ Важно отметить, что приведенные метрики - не медианы, а минимальные значение. Почему - хороший вопрос, но к авторам инструмента.

additionaltestoutput.benchmark.message_ru.tensor.sbis.droid.benchmark.micro.ButtonBenchmarkInflation.inflate_AndroidButton.txt

Files under the "build" folder are generated and should not be edited.

```

1      356,338 ns 303 allocs [Trace](file://ButtonBenchmarkInflation.inflate_AndroidButton_2025

```

ru.tensor.sbis.droid.benchmark.micro.test-benchmarkData.json

Files under the "build" folder are generated and should not be edited.

```

25      "benchmarks": [
26          {
27              "name": "inflate_AndroidButton",
28              "params": {},
29              "className": "ru.tensor.sbis.droid.benchmark.micro.ButtonBenchmarkInflation",
30              "totalRunTimeNs": 3210667770,
31              "metrics": {
32                  "timeNs": {
33                      "minimum": 356338.25,
34                      "maximum": 648799.8522727273,
35                      "median": 451432.7102272727,
36                      "coefficientOfVariation": 0.21087273014626506,
37                      "runs": [
38                          603662.5227272727,
39                          426695.8068181818,
40                          360720.2954545454,
41                          356338.25,
42                          455434.4659090909,
43                          648799.8522727273,
44                          503820.0113636364,
45                          425653.8636363636,
46                          474224.6022727273,
47                          447430.9545454545
48                      ]
49                  },
50                  "allocationCount": {

```

А давайте теперь бенчмаркнем старт контроллер? И тут мы упираемся в то, что код инициализации платформы написан с защитой от повторного вызова.

```

144     init(lib_name, context, type, session_id, enable_crashes: false) { _, action -> action() }
145 }
146
147 @Synchronized
148 @Throws(SbisException::class)
149 fun init(
150     lib_name: String,
151     context: Context,
152     type: AppTypeEnum?,
153     session_id: UUID?,
154     enable_crashes: Boolean,
155     trace: (label: String, action: () -> Unit) -> Unit
156 ) {
157     appPackageName = context.packageName
158     Log.d(appPackageName, msg: "started sbis core initialization")
159     if (!alreadyInitialized) {
160         Log.d(appPackageName, msg: "started loading sbis core libraries")
161         assetManager = context.resources.assets
162         trace(label: "loading $lib_name") {
163             System.loadLibrary(lib_name)
164         }
165     }
166 }

```

И тут выходит на сцену следующий тип бенчмаркинга.

Macrobenchmark

Характерной особенностью данного способа является возможность рестарта процесса - "холодный" запуск приложения. Есть и другие подстройки запуска процесса.

Бенчмарк инициализации контроллера

Накидываем пустое приложение (чтобы исключить влияние остальных компонентов наших приложений), и добавляем туда только инициализацию контроллера.

```

internal class App : Application() {
    override fun onCreate() {
        super.onCreate()
        AppSharedSettings.setSharedSettings(
            AppSettingsImpl(
                ResolverWrapper(applicationContext),
                FileWrapper(applicationContext),
                applicationContext
            )
        )
        trace("Platform#init") {
            Initializer.setAppVersionInfo(
                "25.4100",
                "1"
            )
            Initializer.init(
                "sbis-app-controller",
                context,
                AppTypeEnum.GENERIC_APPLICATION,
                UUID.randomUUID(),
                true
            ) { _, action ->
                action()
            }
        }
        trace("Platform#afterLoadApplication") {

```



```

        Initializer.initAfterLoadApplication()
    }
}
}

```

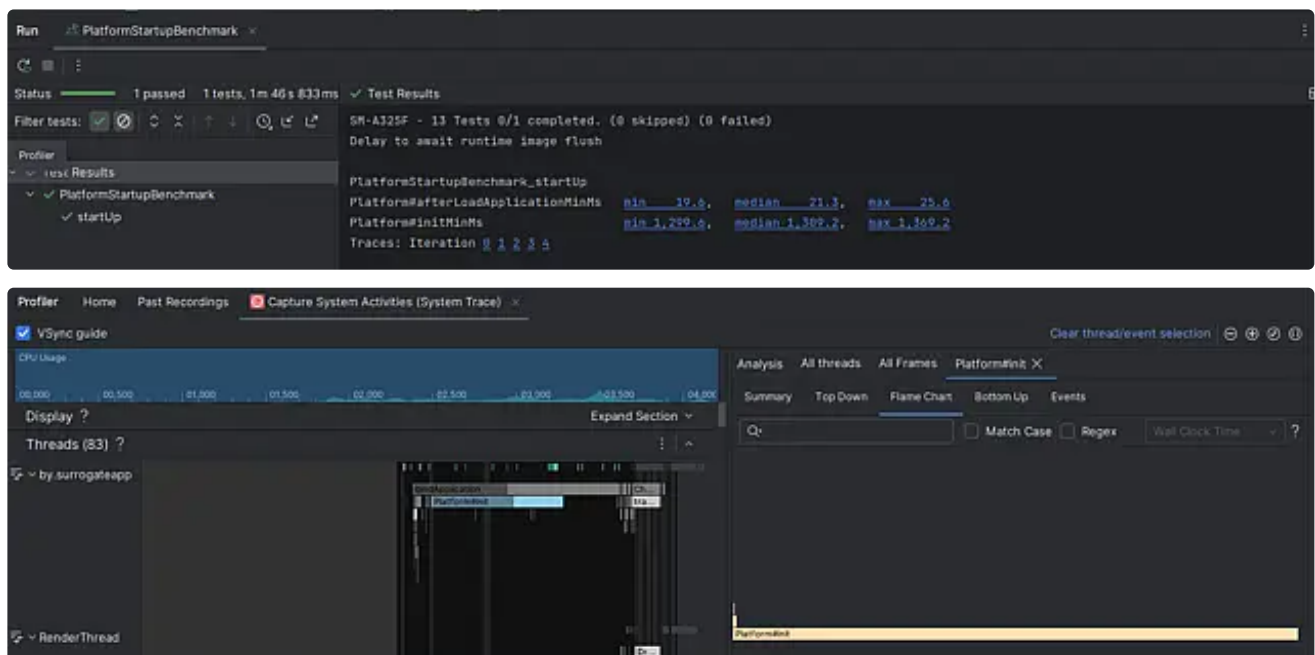
Создаем бенчмарк:

```

@RunWith(AndroidJUnit4::class)
class PlatformStartupBenchmark {
    @get:Rule
    val benchmarkRule = MacrobenchmarkRule()
    @OptIn(ExperimentalMetricApi::class)
    @Test
    fun startup() = benchmarkRule.measureRepeated(
        packageName = "ru.tensor.saby.surrogateapp",
        metrics = listOf(
            TraceSectionMetric(
                "Platform#init",
                mode = TraceSectionMetric.Mode.Min
            ),
            TraceSectionMetric(
                "Platform#afterLoadApplication",
                mode = TraceSectionMetric.Mode.Min
            )
        ),
        iterations = 5,
        startupMode = StartupMode.COLD
    ) {
        pressHome()
        startActivityAndWait()
    }
}

```

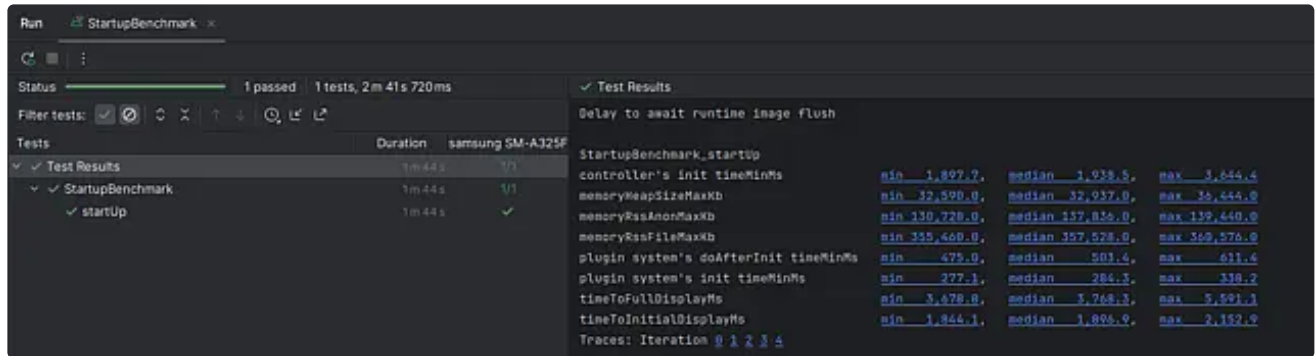
И получаем отчет:



К сожалению, как только мы проваливаемся в *JNI*, перестаем видеть что-либо в *systrace*. Для этого требуется ручное инструментирование. Заметим, что 1.3 сек - половина нашего бюджета производительности на запуск приложения до *MainScreen*.

Бенчмарк приложения

На кошках потренировались. Давайте теперь замерим запуск приложения Saby до экрана авторизации.



Давайте взглянем на профиль, собранный по инициализации плагинной системы:



Особенно маячат 2 плагина: *CommunicatorCommonPlugin* и *TaskCardApiPlugin*. Изначально они не были проинструментированы и мы добавляем *trace*-секции в каждый из них. Начнем с *CommunicatorCommonPlugin*:

```
object CommunicatorCommonPlugin {  
    ...  
    override fun doAfterInitialize() {  
        trace("MessageFailureSubscription") {  
            sendMessageErrorLogger()  
        }  
        trace("AuthSubscription") {  
            subscribeOnLoginLogoutEvent()  
        }  
        trace("FeatureChecker#InitFeatures") {  
            communicationFeatureCheckerImpl.initFeatures(communicationFeatureSet)  
        }  
    }  
}
```

Запускаем бенчмарк-тест и получаем трейс:



Отчетливо видно, что 90% времени занимает *MessageFailureSubscription*. Что же это за код?

```
private fun sendMessageErrorLogger() {  
    // Ссылку на подписку обязательно хранить.  
    msgLoggerSubscription = MsgLogger.instance().onLog().subscribe(  
        object : OnLogCallback() {  
            override fun onEvent(msg: String) {  
                Timber.w(SendMessageException(msg))  
            }  
        }  
    )  
}
```

```

        msgLoggerSubscription.enable()
    }

```

Кто бы мог подумать, что создание инстанса объекта контроллера и вызов на нем метода может занимать столько ресурсов, правда? Давайте ради интереса посмотрим, как устроен метод `MsgLogger.instance()` ([прюф](#)):

```

CJNIEXPORT jobject JNICALL
Java_ru_tensor_sbis_communicator_generated_MsgLogger_instance(JNIEnv* jniEnv,
jobject /*this*/)
{
    try
    {
        DJINNI_FUNCTION_PROLOGUE0(jniEnv);
        auto scoped_ctx = sbis::log::ThreadContext::SetScopedWithMethodName (
sbis::GenerateUUIDRandomDevice(), L"[sbis-communicator-mobile:djinni]
[MsgLogger.Instance]"_sv, 0 );
        auto thread_registrator = ::djinni::ThreadInfoRegistrator(
::djinni::ThreadType::NATIVE );
        std::optional< sbis::ContextEnvironment > ctx_env;
        std::optional< sbis::ScopedContextEnvironment > scoped_ctx_env;
        if( thread_registrator )
        {
            ctx_env.emplace(::sbis::ContextEnvironment::CreateFromCurrentContext());
            scoped_ctx_env.emplace( *ctx_env );
        }
        auto const start_time = sbis::log::LogTimestamp::Now();
        auto log_record = sbis::log::LogRecord::Create(
sbis::log::LogLevel::llDEBUG, start_time, L"MsgLogger.Instance"_sv );
        log_record.SetLogRecordType( sbis::log::RecordType::MethodStart );
        sbis::log::Submit( std::move( log_record ) );
        auto r = ::sbis::mobile::comm::MsgLogger::Instance();
        log_record = sbis::log::LogRecord::Create(
sbis::log::LogLevel::llDEBUG, L"MsgLogger.Instance"_sv );
        log_record.SetLogRecordType( sbis::log::RecordType::MethodFinish );
        log_record.SetExecPeriodFromStart( start_time );
        sbis::log::Submit( std::move( log_record ) );
        if( !r )
        {
            ::sbis::ErrorMsg( L"Метод
::sbis::mobile::comm::MsgLogger::Instance вернул null, но в djinni он не
помечен опцией nullable. Добавьте опцию или проверьте возвращаемое значение"
);
        }
        return
::djinni::release(::sbis::mobile::comm::NativeMsgLogger::fromCpp( jniEnv, r
));
    }
    JNI_TRANSLATE_EXCEPTIONS_RETURN(jniEnv, 0 /* value doesn't matter */)
}

```

Я конечно не специалист по C++, но вроде не выглядит как простая функция.



Никогда! Никогда! Не выполняйте вызовы к **контроллеру** на **MainThread**!

Решением в данном случае может быть запуск подписки в отдельной потоке/корутине без ожидания завершения. Хорошо, с этим плагином разобрались. Идем к *TaskCardApiPlugin*:

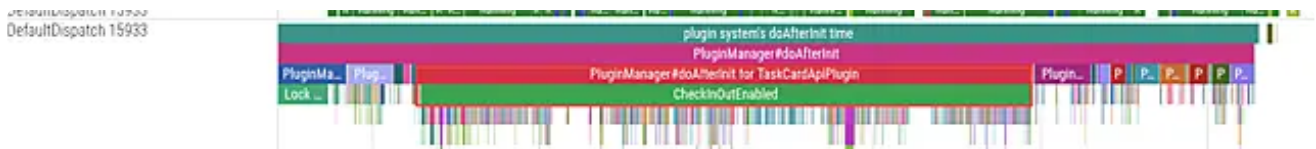
```
object TaskCardApiPlugin {

    override fun doAfterInitialize() {
        docOpenerRegistry?.get()?.run {
            registerAppliedDocCardFactory(TaskAppliedDocCardFactory())

            registerAppliedDocCardFactory(InOutAppliedDocCardFactory(TasksSourceSetKey))
            val isInOutEnabled = trace("CheckInOutEnabled") {
                DocumentPlugin.isInOut
            }
            if (isInOutEnabled)

            registerAppliedDocCardFactory(InOutAppliedDocCardFactory(InOutSourceSetKey))
            registerAppliedDocCardFactory(OtherAppliedDocCardFactory())
        }
    }
}
```

Разметили интересующий нас блок кода и запускаем бенчмарк.



Становится очевидно, что большую часть времени занимает проверка *DocumentPlugin.isInOut*. Что же там такого?

```
TaskCardApiPlugin.kt | DocumentPlugin.kt x
47  object DocumentPlugin : BasePlugin<Unit>() {
88      override val dependency: Dependency = Dependency.Builder()
115      .require(SubdocsFeature::class.java) { subdocsFeatureProvider = it }
116      .build()
117
118      override val customizationOptions = Unit
119
120      val isInOut: Boolean
121      get() = diComponent.dataSources.inout != null
122
123      internal val diComponent: DocumentApplicationComponent by lazy {
124          val defaultRepositoryFactory : DocumentRepositoryFactory = defaultRepositoryFactoryProvider.get()
125          val documentDependency = object : DocumentDependency,
126              AttachmentListViewerFactory by attachmentListViewerFactoryProvider.get(),
127              PersonCardProvider by personCardProvider.get(),
128              LoginInterface.Provider by loginInterfaceProvider.get(),
129              ViewerSliderIntentFactory by viewerSliderIntentFactoryProvider.get(),
130              VerificationFragmentProvider by verificationFragmentProvider.get(),
131              OpenLinkController.Provider by openLinkControllerProvider.get(),
132              WebViewFeature by webViewFeatureProvider.get(),
133              PassageComponentFactory by passageComponentFactoryProvider.get()
```

Всего лишь создание небольшого dagger-графа.

Каким может быть тут решение? Лучше ответственного за компонент сказать сложно. На первый взгляд возникает сомнение, а точно ли хорошая идея, что один плагин явно узнает про другой - ведь это создает жесткую связку между *impl*-частями модулей. Кажется, что логика размазана по двум сущностям. Дает ли это какие-то преимущества не берусь сказать, но явно создает проблему при инициализации плагиновой системы.

На этом предлагаю на сегодня закончить и продолжить в другой раз с новыми силами).

Итоговый анализ

Несмотря на такое обилие инструментов и подходов, улучшение и анализ производительности является нетривиальной задачей. Многие из вас уже сталкивались с ошибками производительности - уверен, что поделившись своим опытом вы поможете коллегам расширить горизонты в этой области.

Ссылки на новости:

<https://online.sbis.ru/news/03c87196-3685-45b9-b626-ec14f0c52514><https://link.sbis.ru/news/84ece6be-40c2-468b-a9e0-e2779b8d41ce>